- Exception handling
- Injecting additional headers into a response
- Custom content renderers
- Support for `async` calls

All else being equal, I'd recommend you start by using Ninja, only moving to other libraries if you have deeper needs that aren't covered. Finally, I'd really like to point out the self-control involved in getting all the way through chapter 11 without making a bad bankruptcy joke.

11.4 Exercises

1. Write an API to list all the `Band` objects in the system with the corresponding `Musician` data nested within each `Band` object.
2. Write an API to update the fields of a `Musician` object, requiring authentication to do so.

Summary

- An API interface can provide access to the data in your project to outside systems.
- Web pages based on the Django Template Engine aren't the only kind of frontend you might build, and by building an API, these other interfaces can use the same logic from your views and the same data in your project.
- *CRUD* is short for *create*, *read*, *update*, and *delete*, the actions you perform on data in your system. This is true whether you're dealing with the objects directly, through the ORM, or indirectly, through the Django Admin or an API.
- REST is a loosely defined protocol built around URLs to define "nouns" in your system and HTTP methods to act as "verbs" to access them.
- REST uses HTTP GET to fetch content, POST to create or modify content, PUT and PATCH to update content, and DELETE to remove content.
- REST does not specify what data is in a call, but two common formats are JSON and XML.
- To use a REST API, you need to know which HTTP methods are used and the format of the payload.
- Versioning an API means including a version number in the base of the API's URL. This can allow you to future-proof breaking changes, giving your users time to continue to use a deprecated version of the API.
- Django Ninja is a third-party library that uses decorators to turn Django view functions into API endpoints.
- In Ninja, you define a single `NinjaAPI` object as a central collector for API endpoints and use its `api.urls` property to register the API's URLs as Django routes.
- API endpoints can be grouped together, using a Ninja `Router` object, with each router registered with the central `NinjaAPI` object. It is common practice to use